

Course Outline: Building a Basic AI Agent in Python

Title: Building a Basic AI Agent in Python **Learnpack:** + Creación de un agente de IA básico en Python
Prerequisite: + Introducción a agentes (Week 8, Day 23) **Target Audience:** Beginner developers who understand what agents are conceptually and have built APIs with FastAPI **Total Lessons:** 15 **Estimated Total Length:** ~7,500 words **Format:** Mixed (20% hands-on)

Course Description

This course takes students from the conceptual understanding of AI agents (covered in the previous learnpack) to actually building one in Python. Students will implement an agent loop, define tools, write agent prompting instructions, and wire everything together into a working system.

The focus is on simplicity: a minimal agent with a clear objective, 2-3 tools, and a well-defined loop. Students will learn what makes a tool well-structured, how to instruct the agent through prompting, and how to avoid the most common implementation mistakes. By the end, students will have built a basic but functional AI agent.

Course Philosophy

This course emphasizes:

- **Code is execution, LLM is reasoning:** The agent loop and tools are Python code; the LLM only decides what to do next.
- **Simple tools, clear instructions:** A tool does one thing. The agent prompt tells the LLM exactly how to use them.
- **Iteration over perfection:** Start with the smallest working agent, then improve.

Course Statistics Summary

Section	Lessons	Estimated Words
00 — Welcome	1	~450
01 — The Agent Loop	3	~1,600
02 — Tools	3	~1,600
03 — Agent Prompting	3	~1,600
04 — Best Practices and Pitfalls	2	~1,000
05 — Assessment	1	~700
06 — What's Next	1	~450
Total	15	~7,400

Section 00: Welcome

00.0 Welcome 📖 (~450 words)

Learning Objectives:

- Identify what this course builds and what tools/knowledge are required
- Connect the conceptual foundation from the previous learnpack to what students will implement here

Content Outline:

- Recap: students already know the five agent components (Objective, Model, Memory, Tools, Loop) and the Observe → Decide → Act cycle
- This course turns those concepts into working Python code
- What students will build: a basic agent with a loop, 2-3 tools, and prompting instructions
- Prerequisites: Python fundamentals, functions, dictionaries, and basic API knowledge (FastAPI)
- Course roadmap: implement the loop, define tools, write agent instructions, learn best practices

Transitions: The next section starts with the core of every agent — the loop.

Key Principles:

- Concepts without implementation are incomplete — this course closes the gap
- The goal is a minimal working agent, not a production system

Examples:

- What students will have by the end: a Python agent that receives a user question, decides which tool to call, executes it, evaluates the result, and returns an answer
 - Analogy: the previous course was the blueprint, this course is the construction
-

Section 01: The Agent Loop

01.0 Observe, Think, Act, Repeat 📖 (~500 words)

Learning Objectives:

- Translate the Observe → Decide → Act cycle into a Python while loop
- Identify the role of each variable in the loop (memory, iteration counter, stop condition)

Content Outline:

- The agent loop in Python: a **while** loop that runs until the agent decides it is done or hits a max iteration limit
- Each iteration has three steps implemented as code:
 - **Observe:** gather the current state (user input + previous results stored in memory)
 - **Think/Reason:** send the current context to the LLM and get a decision back
 - **Act:** execute the decision (call a tool or return a final answer)
- Key variables: **memory** (list of messages/results), **max_iterations** (safety limit), **done** (boolean flag)
- Pseudocode showing the complete loop structure with these variables
- The LLM call returns structured output indicating either a tool to call or a final answer

Transitions: The loop needs boundaries. The next lesson covers how to control when and why the loop stops.

Key Principles:

- The loop is plain Python — the only "magic" is the LLM call inside it
- Memory grows with each iteration, giving the LLM more context to reason with

Examples:

- Pseudocode:

```
memory = [{"role": "user", "content": user_input}]
for i in range(max_iterations):
    response = call_llm(system_prompt, memory)
    if response.type == "final_answer":
        return response.content
    result = execute_tool(response.tool, response.args)
    memory.append({"role": "tool", "content": result})
```

- Tracing one iteration: user asks "What is the weather in Bogotá?" → LLM decides to call `get_weather("Bogotá")` → tool returns "22°C, cloudy" → result added to memory → next iteration: LLM sees the weather data and returns a final answer

01.1 Loop Boundaries 📖 (~500 words)

Learning Objectives:

- Implement a max iteration limit to prevent runaway loops
- Define clear stopping conditions in the agent's logic

Content Outline:

- Why boundaries matter: without them, the agent can loop forever consuming tokens and time (reinforcement from previous learnpack anti-pattern)
- Two types of stopping conditions:
 - **LLM-driven:** the model decides the goal is met and returns a final answer instead of a tool call
 - **Code-driven:** the iteration counter hits `max_iterations`, forcing the loop to end
- What to return when max iterations is reached: a fallback message explaining the agent could not complete the task
- Logging each iteration: print or store the iteration number, the decision made, and the tool called — essential for debugging
- Best practice: start with a low max (5-10 iterations) and increase only if needed

Transitions: Students understand the loop and its limits. The next lesson puts this into practice.

Key Principles:

- The code enforces hard limits; the LLM enforces soft limits (deciding when done)
- Logging is not optional — without it, debugging an agent is guessing

Examples:

- Code-driven stop: `if i == max_iterations - 1: return "I could not complete the task within the allowed steps."`
- Logging: `print(f"Iteration {i}: decided to call {response.tool} with {response.args}")`
- A well-bounded agent: max 5 iterations for a simple weather lookup (usually completes in 2)

01.2 Write a Basic Agent Loop 📖 (~600 words)

Challenge Prompt: Write a Python function called `run_agent` that implements a basic agent loop: it receives a `user_input` string and a `max_iterations` integer, maintains a `memory` list, and loops up to `max_iterations` times calling a provided `fake_llm(memory)` function that returns a dictionary with either `{"type": "final_answer", "content": "..."} or {"type": "tool_call", "tool": "...", "args": "..."} — when the response is a tool call, append the tool name and args as a string to memory and continue; when it is a final answer, return the content.`

Solution Code:

```
def run_agent(user_input: str, max_iterations: int) -> str:
    memory = [user_input]
    for i in range(max_iterations):
        response = fake_llm(memory)
        if response["type"] == "final_answer":
            return response["content"]
        tool_result = f"Called {response['tool']} with {response['args']}"
        memory.append(tool_result)
    return "Max iterations reached."
```

Challenge Code:

```
def run_agent(user_input: str, max_iterations: int) -> str:
    memory = [user_input]
    # Loop up to max_iterations times
    # Call fake_llm(memory) each iteration
    # If response type is "final_answer", return the content
    # If response type is "tool_call", append the tool info to memory
    # If the loop ends without a final answer, return "Max iterations reached."
    pass
```

Section 02: Tools

02.0 What Are Tools? 📖 (~500 words)

Learning Objectives:

- Define what a tool is in the context of an AI agent
- Explain how the agent decides which tool to use

Content Outline:

- A tool is a Python function the agent can call to interact with the outside world
- Tools are how the agent acts — the LLM reasons, tools execute
- The agent does not hardcode which tool to use; the LLM decides based on the current context and the tool descriptions provided in the prompt
- Tool selection: the agent receives a list of available tools (name + description), and the LLM picks the most appropriate one for the current step
- Result injection: after a tool runs, its output is added to the agent's memory so the LLM can use it in the next iteration
- The developer defines the tools; the LLM chooses among them at runtime

Transitions: The next lesson covers how to structure a tool so the agent can use it reliably.

Key Principles:

- A tool is just a function — nothing special about it except that the agent can call it
- The LLM selects tools based on their descriptions, not their code

Examples:

- `get_weather(city: str) -> str` — takes a city, returns the weather
 - `search_web(query: str) -> str` — takes a query, returns a summary of results
 - The agent sees: "Available tools: `get_weather` (returns current weather for a city), `search_web` (searches the internet for a query)" → user asks about weather → LLM picks `get_weather`
-

02.1 Tool Structure 📖 (~500 words)

Learning Objectives:

- Write a tool as a Python function with a clear signature, description, and return value
- Identify the two parts of a tool: the description (for the agent) and the logic (the actual code)

Content Outline:

- Every tool has two parts:
 - **Description/instructions:** a short text explaining what the tool does, what input it expects, and what it returns — this is what the LLM reads to decide whether to use it
 - **Logic:** the Python function body that executes the action
- A good tool description is one sentence: "Returns the current weather for a given city name"
- A good tool function: accepts typed parameters, does one thing, returns a string result
- Tools should be deterministic: same input → same output (from syllabus best practices)
- Keep tools simple: if a tool needs more than 10-15 lines, it is doing too much
- Registering tools: store them in a dictionary mapping tool names to functions so the loop can call them dynamically

Transitions: Students now understand tool structure. The next lesson puts it into practice.

Key Principles:

- The description is as important as the code — a bad description means the agent will misuse the tool
- One tool, one responsibility

Examples:

- Tool definition pattern:

```
tools = {
    "get_weather": {
        "description": "Returns current weather for a city",
        "function": get_weather
    }
}
```

- Good description: "Searches the product catalog by keyword and returns up to 5 results"
- Bad description: "Does stuff with products" — the agent cannot determine when to use this

02.2 Create a Tool (~600 words)

Challenge Prompt: Write a Python function called `get_greeting` that takes a `name` string and a `language` string ("en" or "es"), and returns a greeting in the specified language — then register it in a `tools` dictionary with a `"description"` key and a `"function"` key so an agent loop could look it up by name.

Solution Code:

```
def get_greeting(name: str, language: str) -> str:
    if language == "es":
        return f"Hola, {name}!"
    return f"Hello, {name}!"

tools = {
    "get_greeting": {
        "description": "Returns a greeting for a name in en or es",
        "function": get_greeting
    }
}
```

Challenge Code:

```
def get_greeting(name: str, language: str) -> str:
    # Return "Hola, {name}!" if language is "es"
    # Return "Hello, {name}!" otherwise
    pass

tools = {
```

```
# Register get_greeting with a description and function reference  
}
```

Section 03: Agent Prompting

03.0 Instructing the Agent 📖 (~500 words)

Learning Objectives:

- Write a system prompt that tells the LLM how to behave as an agent
- Structure agent instructions around the Observe, Decide, Act phases

Content Outline:

- Agent prompting: the system prompt is how the developer programs the agent's behavior — it defines the objective, the available tools, and the expected response format
- The three instruction blocks (from syllabus):
 - **Observe:** tell the LLM what information to pay attention to (user input, tool results, memory)
 - **Decide:** tell the LLM how to choose between tools or returning a final answer
 - **Act:** tell the LLM the exact format to respond in (e.g., JSON with **type**, **tool**, **args** fields)
- The response format is critical: if the LLM responds in free text instead of structured JSON, the code cannot parse it
- Including tool descriptions in the prompt: list each tool's name, parameters, and description so the LLM knows what is available
- Keep the prompt concise — long, rambling instructions confuse the model

Transitions: The next lesson covers common mistakes when writing agent prompts.

Key Principles:

- The system prompt is the agent's brain programming — clarity here determines agent quality
- Structured output (JSON) is not optional; the code layer needs to parse the LLM's decisions

Examples:

- System prompt skeleton:

```
You are an agent. Your objective is: [objective].  
Available tools: [list with descriptions].  
Each turn, respond with JSON:  
{  
  "type": "tool_call",  
  "tool": "name",  
  "args": {...}  
}  
or {  
  "type": "final_answer",  
  "content": "..."  
}
```

- Good instruction: "If the user's question can be answered with the information already in the conversation, respond with a final_answer. Otherwise, call the most relevant tool."
- Including tools: "get_weather(city: str) — returns current weather for the given city"

03.1 Common Prompting Mistakes 📖 (~500 words)

Learning Objectives:

- Recognize the anti-pattern of overly complex agent prompts
- Identify how vague or contradictory instructions lead to unpredictable agent behavior

Content Outline:

- Anti-pattern: prompt too complex (from syllabus) — when the system prompt is 2 pages long with dozens of rules, the LLM loses focus and makes inconsistent decisions
- Symptom: the agent ignores some instructions, follows others randomly, or produces malformed responses
- Anti-pattern: vague decision criteria — "use the best tool" does not help the LLM; "if the user asks about weather, use `get_weather`; if the user asks a general question, use `search_web`" does
- Anti-pattern: not specifying the response format clearly — the LLM defaults to natural language instead of JSON, and the parsing breaks
- Anti-pattern: contradictory instructions — "always use a tool" combined with "respond directly if you know the answer" creates confusion
- How to fix: keep the prompt under 300 words, use explicit if/then rules, test with edge cases, and iterate

Transitions: Students now know good and bad prompting practices. The next lesson lets them write agent instructions.

Key Principles:

- Shorter, clearer prompts produce more reliable agents
- Every instruction should be testable: if you cannot verify whether the agent followed it, remove it

Examples:

- Bad prompt: "You are a super intelligent agent that can do anything. Be helpful, creative, and accurate. Use tools wisely and respond in a structured way."
- Good prompt: "You are a weather assistant. You have one tool: `get_weather(city)`. If the user asks about weather, call `get_weather`. Otherwise, respond with a `final_answer`. Always respond in JSON format."
- Contradictory: "Never call a tool more than once" + "Keep trying until you find the answer" — the agent cannot satisfy both

03.2 Write Agent Instructions 📖 (~600 words)

Challenge Prompt: Write a Python function called `build_system_prompt` that takes a `tools` dictionary (where each key is a tool name and each value has a `"description"` string) and an `objective` string, and returns a formatted system prompt string that includes the objective, lists each tool with its description, and instructs the LLM to respond in JSON with either `{"type": "tool_call", "tool": "...", "args": {...}}` or `{"type": "final_answer", "content": "..."}.`

Solution Code:


```
def build_system_prompt(tools: dict, objective: str) -> str:
    tool_lines = ""
    for name, info in tools.items():
        tool_lines += f"- {name}: {info['description']}\n"
    return (
        f"You are an agent. Objective: {objective}\n"
        f"Available tools:\n{tool_lines}"
        f"Respond with JSON:\n"
        f'{{{"type": "tool_call", "tool": "name", "args": {}}}}\n'
        f'or {{{"type": "final_answer", "content": "..."}}}'
    )
```

Challenge Code:

```
def build_system_prompt(tools: dict, objective: str) -> str:
    # Build a string listing each tool name and description
    # Return a system prompt that includes:
    #   - The objective
    #   - The tool list
    #   - JSON response format instructions
    pass
```

Section 04: Best Practices and Pitfalls

04.0 Agent Best Practices 📖 (~500 words)

Learning Objectives:

- Apply practical guidelines for building reliable agents
- Prioritize simplicity and observability in agent design

Content Outline:

- **Keep tools simple and specific:** each tool does one thing with a clear description (from syllabus best practices)
- **Limit tools to 2-4 per agent:** more tools means more confusion for the LLM (from syllabus best practices)
- **Validate inputs:** before executing a tool, check that the arguments are valid — do not blindly trust the LLM's output
- **Limit iterations:** always set a `max_iterations` value; start low (5-10) and increase only if tasks genuinely need more steps
- **Log every decision:** record what the agent decided, which tool it called, and what result it got — debugging without logs is impossible (from syllabus best practices)
- **Design tools to be deterministic:** same input should produce the same output, making the agent's behavior predictable (from syllabus best practices)
- Summary: simple tools + clear prompt + bounded loop + logging = reliable agent

Transitions: The next lesson covers the flip side — the most common errors students will encounter.

Key Principles:

- Reliability beats cleverness — a simple agent that works is better than a complex one that sometimes works
- Observability (logging) is what separates a debuggable agent from a black box

Examples:

- Input validation: if the LLM returns `{"tool": "get_weather", "args": {"city": ""}}`, the code should catch the empty string before calling the tool
 - Logging: `print(f"[Iteration {i}] Tool: {tool_name}, Args: {args}, Result: {result[:100]}")`
 - Deterministic tool: `get_weather("Bogotá")` always returns the current weather from the same API — no randomness in the tool itself
-

04.1 Common Errors 📖 (~500 words)

Learning Objectives:

- Identify the most frequent implementation mistakes when building agents
- Recognize symptoms of each error in a running agent

Content Outline:

- **Too much logic inside a tool:** a tool that searches, filters, sorts, and formats is doing the agent's job — keep tools atomic and let the loop orchestrate (from syllabus anti-patterns)
- **Ambiguous tool descriptions:** if two tools sound similar, the LLM picks randomly — make descriptions mutually exclusive (from syllabus anti-patterns)
- **No stopping condition:** the agent loops until max iterations every time because the prompt does not clearly define what "done" looks like (from syllabus anti-patterns)
- **Not validating LLM output:** the LLM sometimes returns malformed JSON, invents tool names that do not exist, or provides wrong argument types — the code must handle all of these gracefully
- **Ignoring tool results:** appending a tool result to memory but not prompting the LLM to evaluate it means the agent calls the same tool repeatedly without progress
- How to detect each error: symptoms and what to look for in the logs

Transitions: Students have the full picture — loop, tools, prompting, best practices, and pitfalls. The next section tests their understanding.

Key Principles:

- Most agent failures are not LLM failures — they are engineering failures (bad prompts, missing validation, poor tool design)
- When an agent misbehaves, check the logs before blaming the model

Examples:

- Ambiguous tools: `search_info` and `find_data` — the LLM cannot tell them apart. Fix: rename to `search_product_catalog` and `lookup_user_profile`
 - Malformed output: LLM returns `I'll call get_weather for Bogotá` instead of JSON — the parser crashes. Fix: wrap parsing in try/except and re-prompt if invalid
 - Repeated tool calls: agent calls `search_web("hotels")` three times in a row because memory has the result but the prompt does not say "if you already have search results, analyze them instead of searching again"
-

Section 05: Assessment

05.0 Knowledge Check 🧠 (~700 words)

Learning Objectives:

- Demonstrate ability to implement and debug a basic agent loop
- Identify correct tool structure and agent prompting patterns
- Recognize implementation anti-patterns

Question Format: Scenario-based

Questions:

Question 1: A student writes an agent loop that calls `fake_llm(memory)` but never appends the tool result to `memory` before the next iteration. What will happen?

Expected reasoning: The LLM will not see the tool's output, so it will keep making the same decision over and over. The agent will either loop until max iterations calling the same tool repeatedly, or the LLM will give a final answer without ever using the tool result. The fix is to append each tool result to memory after execution.

Question 2: A developer defines a tool like this:

```
def do_research(topic):  
    results = search_web(topic)  
    filtered = filter_relevant(results)  
    summary = summarize(filtered)  
    return format_report(summary)
```

What anti-pattern does this represent?

Expected reasoning: Too much logic inside a single tool. This tool searches, filters, summarizes, and formats — it should be broken into separate tools so the agent can orchestrate each step. The agent loop exists precisely to chain simple actions together.

Question 3: An agent has this system prompt: "You are a helpful assistant. Use tools when needed. Respond however you like." The agent frequently returns plain text instead of JSON, crashing the parser. What is the problem?

Expected reasoning: The prompt does not specify the response format. The LLM defaults to natural language because no JSON structure was defined. The fix is to explicitly instruct the LLM to respond only in JSON with the

exact schema the code expects.

Question 4: An agent has two tools: `search_articles(query)` described as "Finds articles about a topic" and `find_content(query)` described as "Looks for content about a topic." The agent frequently picks the wrong one. Why?

Expected reasoning: The tool descriptions are ambiguous — they sound almost identical. The LLM cannot reliably distinguish between them. The fix is to make descriptions mutually exclusive, e.g., "Searches news articles from the last 7 days" vs. "Searches the internal knowledge base for documentation."

Question 5: A student's agent works for simple questions (1-2 tool calls) but fails on complex ones. The `max_iterations` is set to 3. What should the student do?

Expected reasoning: The max_iterations is too low for complex tasks. The student should increase it (e.g., to 8-10) while keeping it bounded. The principle is to start low and increase based on observed needs, not to remove the limit entirely.

Question 6: An agent's tool is defined correctly and works when called manually, but the agent never calls it. The tool is registered in the `tools` dictionary. What is the most likely cause?

Expected reasoning: The tool description is either missing from the system prompt or is unclear. The LLM does not know the tool exists or does not understand when to use it. The fix is to include the tool name and description in the system prompt so the LLM can see it as an available option.

Evaluation Criteria:

- Students should demonstrate understanding of the agent loop mechanics (memory, iteration, stopping)
- Students should identify tool structure issues (too much logic, ambiguous descriptions)
- Students should recognize prompting failures (missing format, vague instructions)

Score Interpretation:

- 6/6: Ready to build agents independently
- 4-5/6: Solid understanding with minor gaps — review the missed concept
- Below 4/6: Revisit tool structure and agent prompting sections

Section 06: What's Next

06.0 Your First Agent is Ready (~450 words)

Content Outline:

- Recap: students can now implement an agent loop in Python, define and register tools, write system prompts that instruct the LLM, and avoid the most common pitfalls
- What they built: a minimal agent with a loop, tools, structured prompting, input validation, and logging
- Where this leads: future skills will build on this foundation — connecting agents to real APIs, adding persistent memory, working with more sophisticated tool systems, and eventually building multi-agent systems
- The next skill (Day 24) covers working with files in Python — a practical capability that can become a tool in future agents

- Encouragement: the agent pattern students just learned is the same pattern used in production AI systems — the complexity grows, but the core loop stays the same
- Reminder: simple agents with clear goals, few tools, and bounded loops are almost always better than complex ones

Note: This is conclusion only — no theory, exercises, or assessment.